PORTFOLIO

PORTFOLIO

임찬진

플레이어들에게 행복한 시간을 주고싶은 개발자

포트폴리오 목차

Project	Page	사용 언어	사용 엔진
S.T.L	**Page 3 - 10**	**C++20**	**D3D12**
└ 논문 발표	**Page 11**	-	-
└ Deferred 렌더러	**Page 12 - 14**	**C++**	**D3D12**
TheGreatestNature	**Page 15**	-	**UE5**
ASSASSIN VS MUTANT	**Page 16**	-	**UE5**
2DGP Curvy	**Page 17**	**Python**	-
FlickDom	**Page 18 - 19**	**C#**	**Unity 6**

Other Activities

AI 활용 능력	**Page 20 - 22**	-	-
마무리	**Page 23**	-	-

S.T.L

GITHUB VIDEO

Team Project



프로젝트	S.T.L (Slay The Lord)
장르	4인 협동 멀티 액션 RPG
설명	파티와 함께 퀘스트를 수행하고 협동 플레이로 보스 몬스터를 공략하는 다크 판타지 게임
개발 인원	3명
담당 역할	glTF 리소스 파싱, 텍스처 URI 해석, MaterialInfo 매핑, D3D12 PBR 렌더링
개발 언어	C++20, STL, HLSL
사용 툴 & 라이브러리	Visual Studio 2022, VS Code, DirectX 12, ImGui, Json, Git
제작 기간	2025.06.23 ~ 2026.08.09

게임 콘텐츠 & 시스템

협동 액션 RPG 전투 구조

4인 파티 전투 흐름에 맞춰 보스 공략, 타깃 전환, 실시간 판정 시스템을 구성했습니다.

- 파티 단위 보스 공략 루프 설계
- 어그로 수치 기반 타깃 전환 처리
- 대시, 가드, 타격 판정을 프레임 단위로 동기화







월드 구성 렌더링

Unreal 배치 데이터 기반 맵 구성

Unreal에서 구성한 Mesh와 Transform 데이터를 자체 엔진의 월드 좌표계로 변환했습니다.

- glTF의 Mesh Primitive와 Material 데이터를 파싱
- Transform 값을 엔진 좌표와 스케일 기준으로 변환
- 메쉬를 월드 위치에 배치해 렌더링

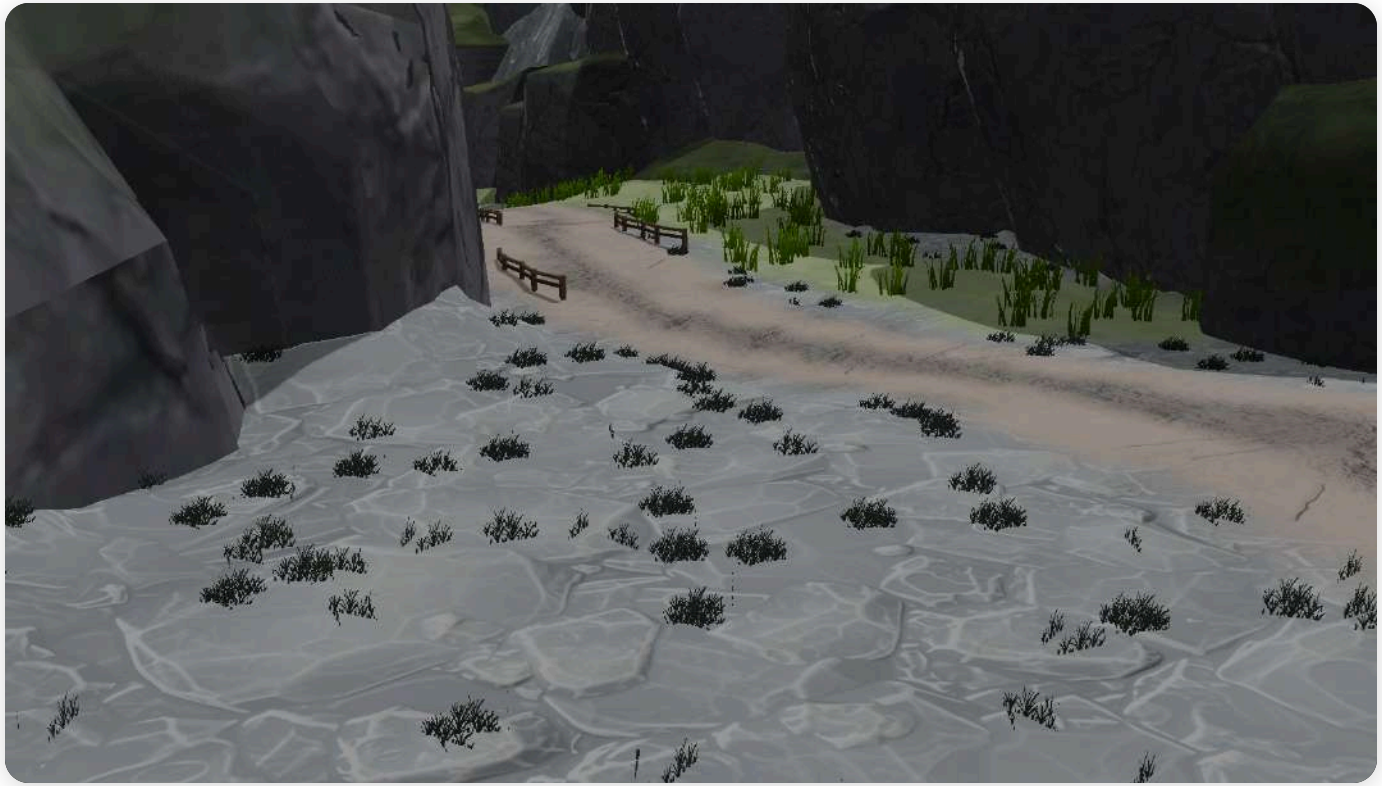


Landscape 텍스처 매핑

raw 파일 기반 지형 텍스처 매핑

Landscape의 텍스처 위치 정보를 raw 파일로 추출해 지형 타일별 레이어로 해석했습니다.

- raw 값을 지형 타일과 텍스처 레이어 데이터로 변환
- 타일별 raw 값에 맞춰 지형 텍스처 선택
- 텍스처 레이어를 자체 엔진 지형 렌더링에 적용



glTF PBR 코드 구현

glTF Material → PBR Shader

glTF Material 데이터를 엔진용 MaterialInfo로 변환하고 D3D12 바인딩을 통해 PBR 셰이더로 전달했습니다.

- **images / textures 인덱스를 따라 텍스처 URI를 해석**
- **base_path와 URI를 조합해 dds 텍스처 경로 복원**
- **Material 상수와 SRV를 PBR 셰이더에 바인딩**

01
URI Resolve

images / textures 인덱스 기반 경로 해석

material이 참조하는 텍스처 URI를 glTF 인덱스 순서대로 해석했습니다.

```
images_uris → texture_source_uris
```

02
Texture Load

상대 URI를 dds 파일 경로로 복원

base_path와 URI를 조합해 텍스처 로더가 읽을 경로로 변환했습니다.

```
base_path / uri → load_texture()
```

03
Shader Bind

Material 상수와 SRV 바인딩

factor 값과 텍스처 플래그를 넘겨 HLSL 샘플링 분기에 사용했습니다.

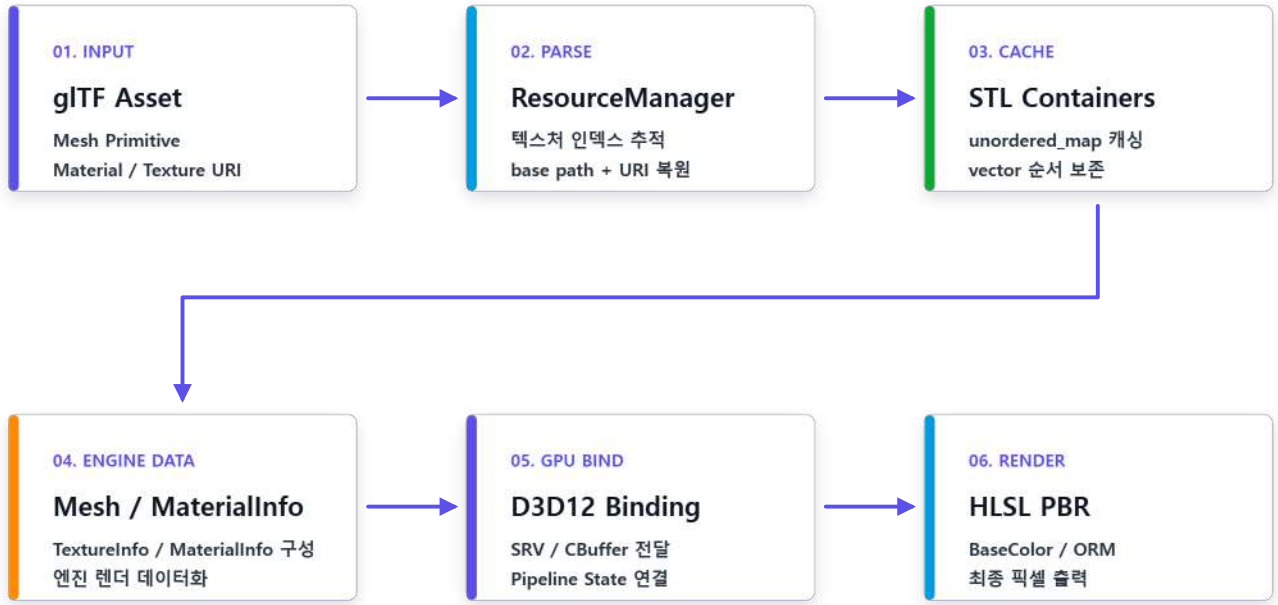
```
bind_material() → PS_GLTF()
```

중요 코드 설명 1

리소스 전달 구조

Unreal에서 추출한 glTF의 images/textures/materials 데이터를 C++20과 STL로 추적해 dds 텍스처 경로와 머티리얼 슬롯을 엔진 데이터로 변환했습니다.

unordered_map 캐싱	문자열 키 기반 캐시로 리소스 중복 로드 방지
vector URI 배열	glTF 인덱스 순서를 유지해 images와 textures 연결 순서 보존
filesystem 경로 조합	모델 기준 상대 URI를 실제 dds 경로로 복원



핵심 | 문자열 키 캐싱 → glTF 순서 보존 → 경로 복원 → 엔진 데이터 변환 → GPU 바인딩 → PBR 렌더링

중요 코드 설명 2

Shader 활용 포인트

ResourceManager가 채운 Material 상수와 텍스처 플래그를 HLSL에서 사용해 PBR 기본값과 ORM 샘플링 값을 같은 경로로 처리했습니다.

CPU to GPU	**CPU MaterialInfo를 GPU cbMaterial에 매핑**
Texture Flag	**ORM 텍스처 유무로 factor와 샘플링 값을 분기**
Final Color	**Lighting/IBL/Emissive에 Phantom Aura를 합산**

01. cbMaterial Structure

Gltf_Shader.hlsl

```
cbuffer cbMaterial : register(b2)
{
  float4 BaseColorFactor;
  float4 EmissiveAndMetallicFactor;
  #define EmissiveFactor EmissiveAndMetallicFactor.rgb
  #define MetallicFactor EmissiveAndMetallicFactor.a

  float RoughnessFactor;
  int HasMetallicRoughnessTexture;
  ....
};
```

02. Material Value Usage

Gltf_Shader.hlsl

```
roughness = lerp(RoughnessFactor, orm.g * RoughnessFactor, hasMetTex);
metallic = lerp(MetallicFactor, orm.b * MetallicFactor, hasMetTex);
float3 emissive = finalEmissive.rgb;

float4 litColor = Lighting(..., albedo, metallic, roughness, ao);
float3 iblColor = CalculateIBL(N, V, albedo, metallic, roughness, ao);
finalColor = (litColor.rgb * shadowFactor) + iblColor + finalEmissive;

if (g_otherplayerid > -1)
  finalColor = ApplyPhantomAura(finalColor, N, V);
```

개발 결과 및 성과

결과

- D3D12 기반 자체 렌더링 파이프라인 구성
- glTF 에셋을 실제 게임 화면에 안정적으로 출력
- BaseColor, Normal, ORM, Emissive 텍스처를 통합한 PBR 셰이더 구조 구현

성과

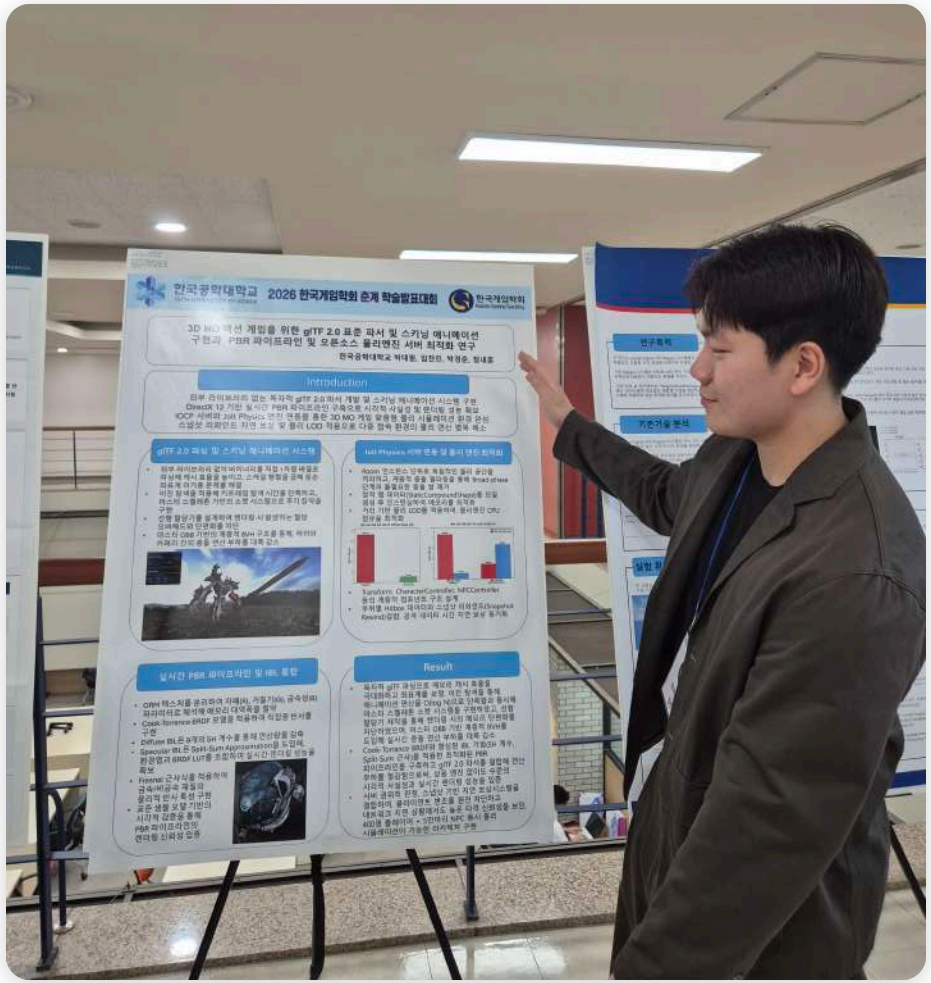
- 그래픽스 구현 내용을 논문 주제와 연결해 기술 구현 과정 문서화
- 졸업작품 우수작 선정 및 특성화장학금 수상
- 한국게임학회 추계학술대회 포스터 논문 전시



논문 발표로 이어지는 그래픽스 구현

S.T.L에서 직접 구현한 glTF 파서, PBR 셰이더, D3D12 렌더링 파이프라인을 기술 주제로 정리했고, 다음 페이지에서 논문 발표 내용으로 연결됩니다.

논문 발표



한국게임학회 춘계 학술발표대회

논문명 : 3D MO 액션 게임을 위한 glTF 2.0 표준 파서 및 스키닝 애니메이션 구현과 PBR 파이프라인 및 오픈소스 물리엔진 서버 최적화 연구

담당 파트 : 그래픽스 구현
glTF 2.0 표준 파서와 PBR 파이프라인 구현 내용을 중심으로 정리했습니다.

핵심 구현
외부 라이브러리 의존을 줄이고 DirectX 12 환경에서 메시와 머티리얼 데이터를 직접 처리했습니다.

PBR 파이프라인
ORM 텍스처, Cook-Torrance BRDF, IBL, Fresnel 기반 표현을 활용해 실시간 렌더링 품질을 개선했습니다.

S.T.L 후속 개발 / C++ · DIRECTX 12

Forward에서 Deferred로: 렌더링 파이프라인 확장

GITHUB

Personal Project

졸업작품 S.T.L의 Forward 렌더링을 기반으로 C++ / DirectX 12 후속 렌더러에 Deferred 구조를 설계·구현했습니다.

Mesh → Geometry Pass → G-buffer → Lighting Pass → 최종 화면



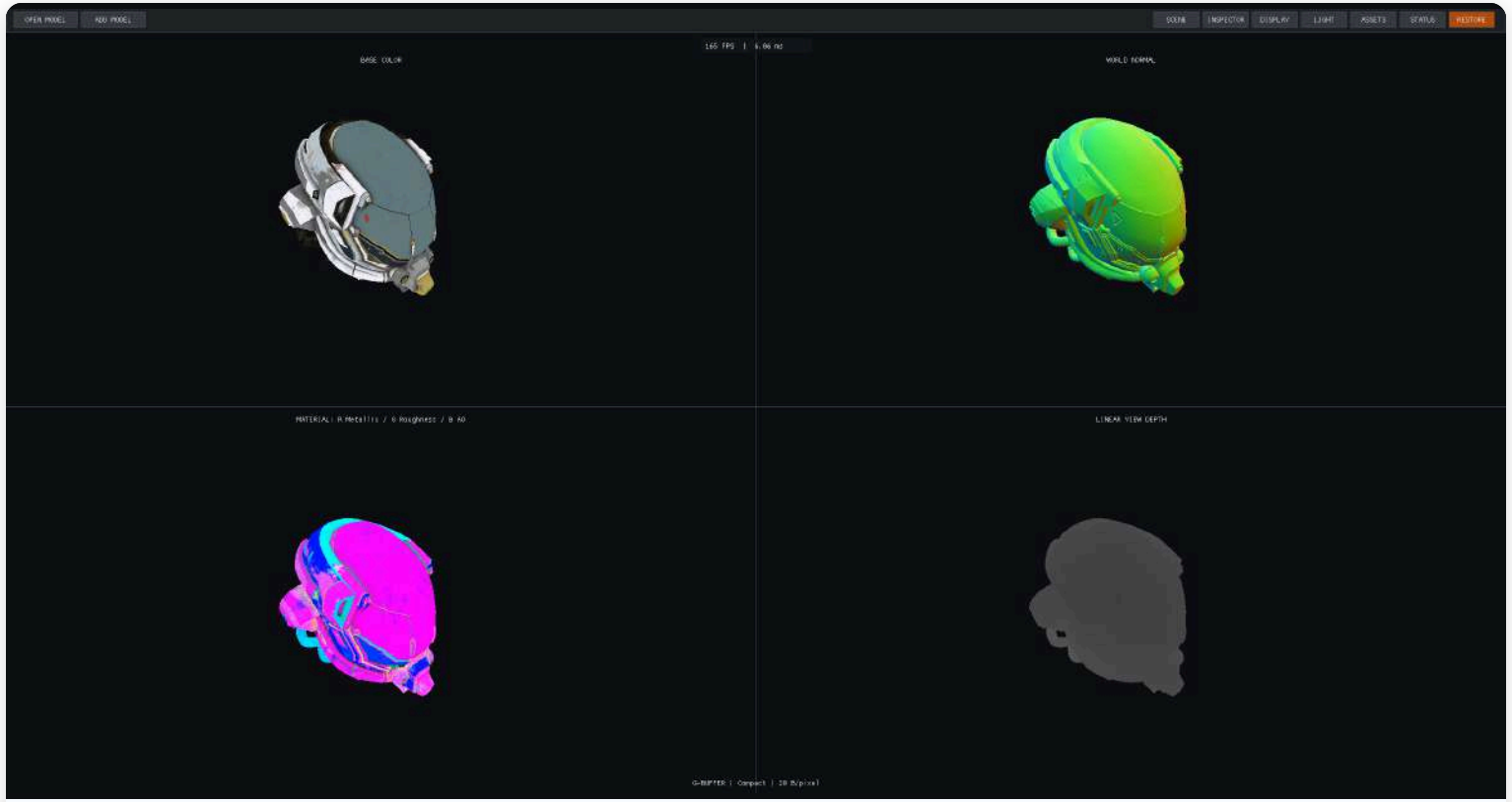
Deferred Lighting 결과 · PBR 직접광 + IBL · Damaged Helmet 외부 샘플 에셋

표면 정보와 조명 계산 분리

Geometry Pass에서 만든 G-buffer를 Lighting Pass가 SRV로 읽어 PBR 직접광과 IBL을 계산했습니다.

원본 게임 전체를 Deferred로 전환한 것이 아니라, S.T.L의 Forward 렌더링 경험을 바탕으로 별도의 후속 렌더러를 구현한 작업입니다.

G-buffer에 기록한 표면 정보



실제 G-buffer를 읽어 구성한 확인 화면 · 원본 재질 텍스처 미리보기가 아님

Geometry Pass 출력

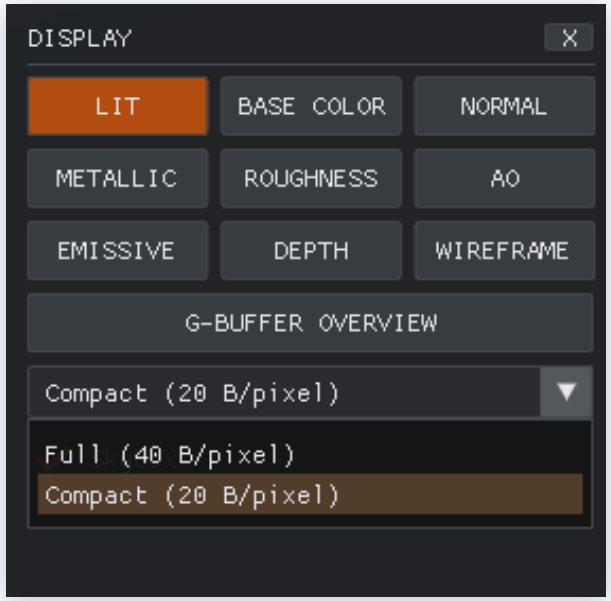
메쉬의 표면 정보를 여러 렌더 타깃에 기록하고, Lighting Pass에서 다시 읽도록 구성했습니다.

- **좌상 · Base Color**
- **우상 · World Normal**
- **좌하 · Material**
  R: Metallic / G: Roughness / B: AO
- **우하 · 시각화용 Linear View Depth**

4분할은 표시 구성일 뿐 실제 G-buffer 타깃 수를 뜻하지 않습니다. Emissive는 저장하지만 이 화면에는 표시하지 않았습니다.

Deferred 구조 확인과 저장 비용 개선

확인용 뷰어



실행 중 Lit 결과와 G-buffer 채널을 전환하고 Full / Compact 저장 모드를 비교했습니다.

패스 간 리소스 관리

Geometry → Lighting 실행 순서를 구성하고, 렌더 타깃 쓰기 상태에서 셰이더 읽기 상태로 전환되는 리소스 배리어를 관리했습니다.

G-buffer 저장 구조 개선

Compact 모드는 저장 포맷을 줄이고, 깊이와 역 View-Projection 행렬로 월드 위치를 복원합니다.

Full 40 B/pixel → Compact 20 B/pixel

학습 결과

표면 정보와 조명 계산의 분리, 패스 간 데이터 전달, 저장 비용과 위치 복원 방식을 직접 구현했습니다. 50% 감소는 G-buffer 색상 타깃의 픽셀당 데이터 용량 기준이며 전체 VRAM이나 GPU 실행 시간 수치가 아닙니다.

TheGreatestNature

GITHUB VIDEO

Team Project









프로젝트	TheGreatestNature
장르	단편 시네마틱 영상
설명	자연을 바라보며 치유되는 감정을 MetaHuman과 Sequencer로 연출한 UE5 시네마틱
개발 인원	팀 프로젝트
담당 역할	MetaHuman 리타게팅, 표정 제어, Camera Rail, Sequencer 컷 편집, 조명 구성
개발 언어	Blueprint
사용 툴 & 라이브러리	Unreal Engine 5, MetaHuman, Sequencer, Git
제작 기간	2025.11.11 ~ 2025.12.16

ASSASSIN VS MUTANT

GITHUB VIDEO

Personal Project









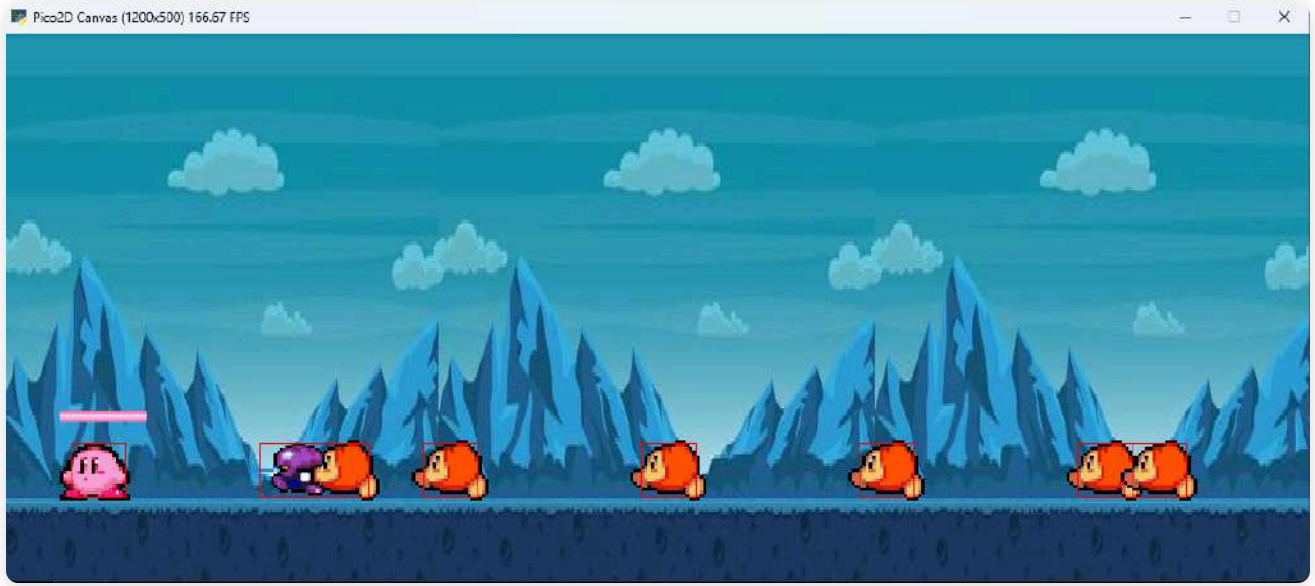
프로젝트	ASSASSIN VS MUTANT
장르	3인칭 3D 액션 게임
설명	암살자 캐릭터가 돌연변이 적을 상대하는 UE5 기반 액션 전투 프로젝트
개발 인원	1명
담당 역할	전투 액션, 무기 스위칭, 적 AI, UI, 승패 흐름, 연출 이미지 적용
개발 언어	Blueprint
사용 툴 & 라이브러리	Unreal Engine 5, Git
제작 기간	2025.06.01 ~ 2025.06.18

2DGP Curvy

GITHUB VIDEO

Personal Project





프로젝트	2DGP Curvy
장르	2D 횡스크롤 액션 게임
설명	Python 기반으로 게임 루프, 충돌 처리, 캐릭터 상태 전이를 직접 구현한 개인 프로젝트
개발 인원	1명
담당 역할	플레이어 조작, FSM, 공격 판정, 몬스터 이동, 씬 전환 구조 구현
개발 언어	Python
사용 툴 & 라이브러리	pico2D, VS Code, Git
제작 기간	2022.10.02 ~ 2022.12.06

FlickDom (AI VIBE CODING)

GITHUB VIDEO

Team Project



프로젝트	FlickDom (AI VIBE CODING)
장르	1 VS 1 전략 플릭 보드게임
설명	디스크를 튕겨 보드 칸을 차지하고 카드 패턴을 완성해 점수를 겨루는 물리 기반 파티 게임
개발 인원	팀 프로젝트
담당 역할	Unity MCP 기반 구현 반복, 멀티플레이 흐름, 사운드 제작 및 적용, 기획 총괄
개발 언어	C#
사용 IDE	Unity 6 Editor, VS Code, Blender
AI 모델	GPT-5 Codex, Unity MCP, Blender MCP, VARCO AI Sound
제작 기간	2026.07.01 ~ 2026.08.10

게임 시스템



목표

3개의 디스크로 보드 칸을 점령하고 카드 패턴을 완성합니다.



조작

디스크를 선택한 뒤 드래그 방향과 힘으로 발사해 다음 턴 전략을 만듭니다.



멀티플레이

Host가 방을 생성하고 Join Code를 공유하면 Client가 접속합니다.



시작 전 준비

Host / Client 역할 확인



Join Code 입력

호스트 코드로 같은 방 접속



상태 표시

접속 후 역할 상태 확인



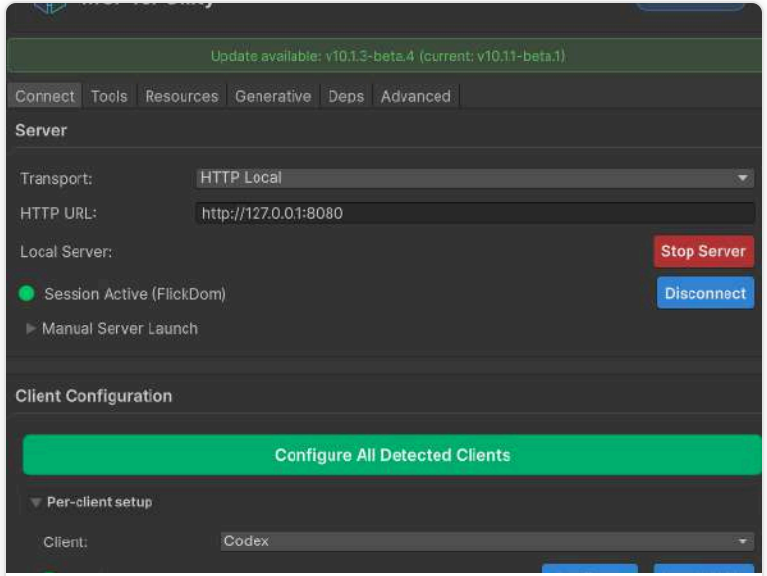
초대 코드 전달

최신 코드 공유 후 시작

AI/MCP 활용으로 이어지는 제작 과정

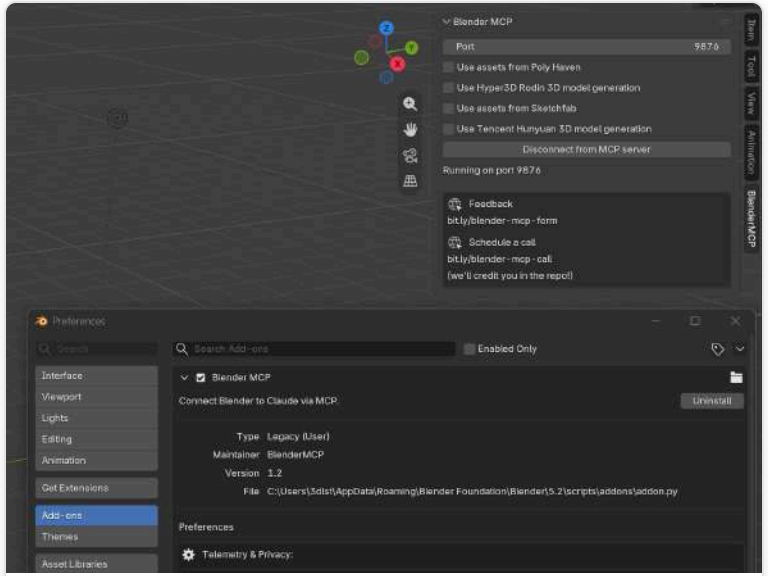
Unity MCP, Blender MCP, VARCO AI Sound를 함께 활용해 구현 반복과 리소스 제작 속도를 높였습니다.

AI/MCP 기반 개발



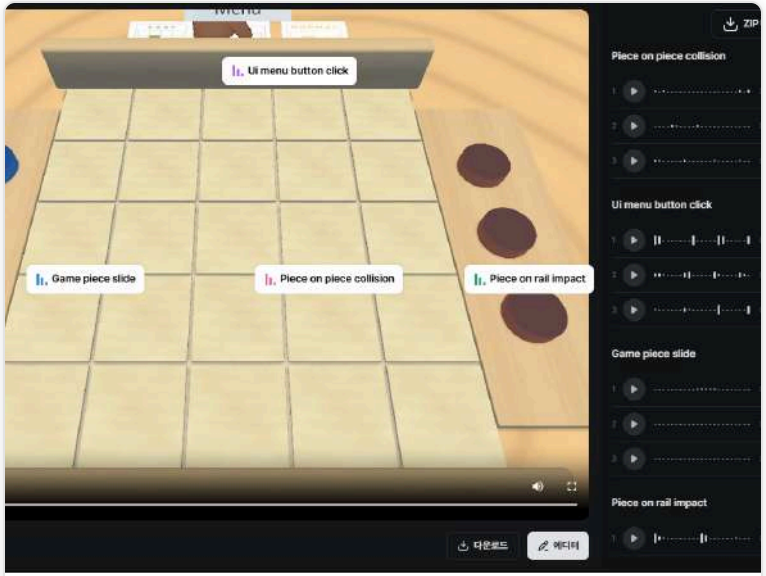
Unity MCP

Unity Editor와 AI 에이전트를 연결해 씬, 오브젝트, 컴포넌트 작업을 빠르게 확인하고 반복했습니다.



Blender MCP

보드, 디스크, 소품 리소스의 비율과 배치 목적을 정리하며 3D 제작 과정을 보조했습니다.



VARCO AI Sound

버튼 클릭, 점수 획득, 실패, 승리 등 상황별 효과음을 제작해 플레이어 피드백을 강화했습니다.

MD 기반 프로젝트 뼈대

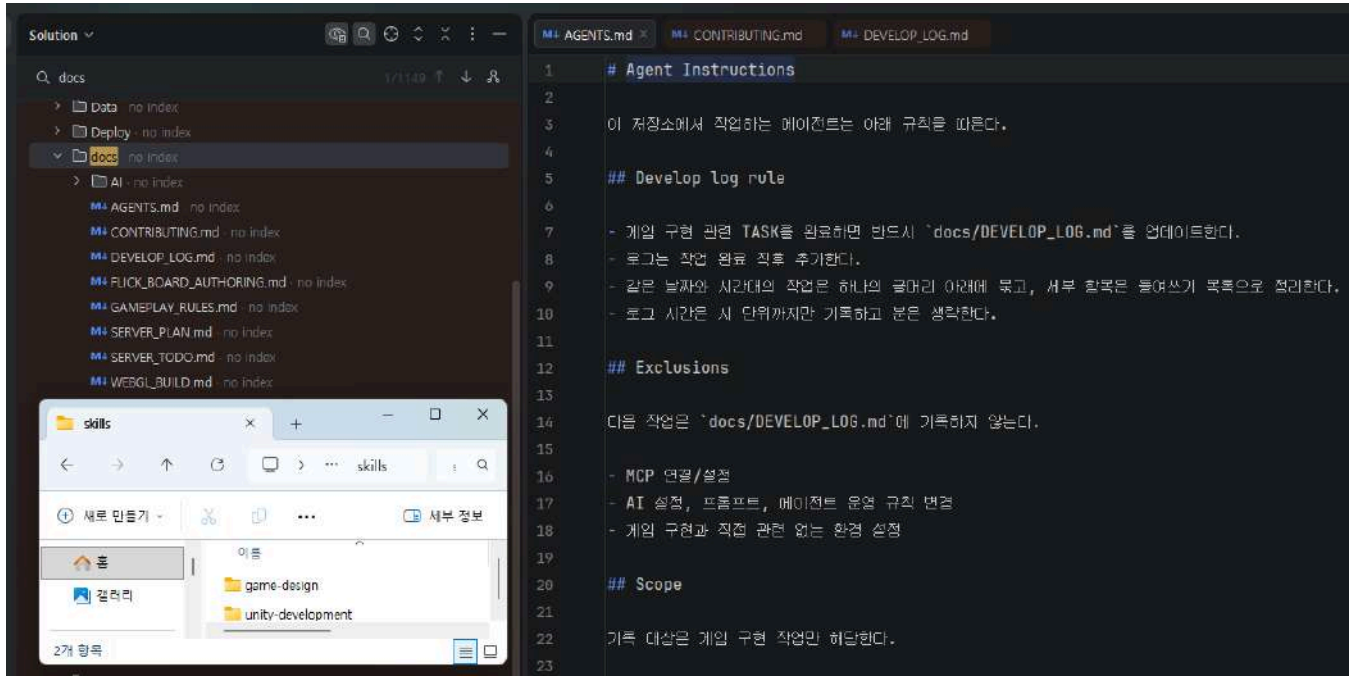
MD 기반 프로젝트 뼈대 및 모듈화

FlickDom 전용 Architecture.md에는 실제 프로젝트 구조와 작업 규칙을 정리했습니다.
기획, 구현, 빌드 문서를 기능별로 분리해 AI(MCP)가 작업 맥락을 빠르게 파악하도록 구성했습니다.

작업 기준 :
Architecture.md에 FlickDom의 실제 폴더 구조와 시스템 기준 정리

기획 모듈 :
GAMEPLAY_RULES.md로 핵심 규칙, SERVER_PLAN.md로 동기화 계획 분리

제작 및 빌드 기준 :
리소스 제작 기준과 WebGL 배포 절차를 별도 문서로 관리



게임 제작 기본 MD 파일

GITHUB

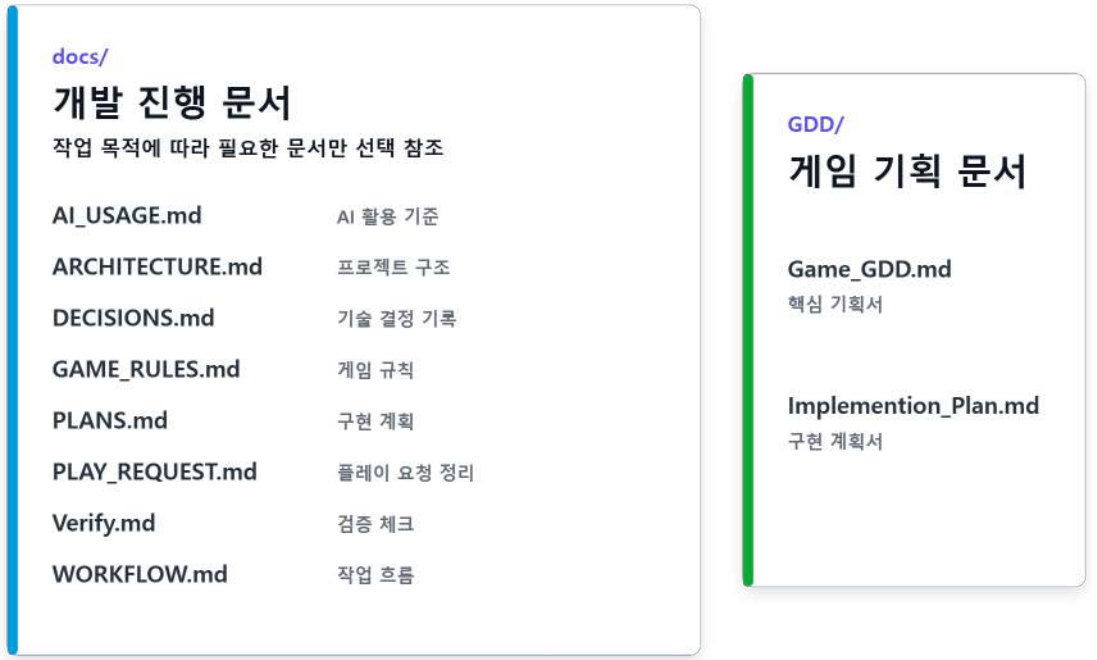
새로운 MD 참조 구조

FlickDom에서는 MD 파일 간 연결이 강해 작업마다 참조 범위가 넓어지고 AI 사용 토큰이 빠르게 소모되었습니다.
새로운 MD 구조는 문서 간 연결 강제를 줄이고, 요청 목적에 맞는 하위 문서만 선택적으로 참조하도록 구성했습니다.

문제 인식 :
MD 간 연결이 강할수록 작업별 참조 범위가 과도하게 확장

읽기 방식 수정 :
요청 목적에 맞춰 docs/, GDD/ 하위 파일 중 필요한 문서만 선택

다음 프로젝트 적용 예정 :
기획, 구현, 검증 문서를 목적별로 분리해 읽는 방식으로 사용 예정



CONTACT

감사합니다.

이름	임찬진
전화번호	010-4032-9600
이메일	ckswlsdl0916@gmail.com